

# Dokumentation: Datenaufbereitung Bodenfeuchte-Vorhersage

Softwarepraktikum Smart Gardening · ML-Komponente · Enes Korkmaz

Sep 29, 2026

## Was das Skript macht

Mit `ML_Bodenfeuchte.py` lese ich die Sensordaten der drei Versuchsbeete ein und bereite sie so auf, dass ich sie für ein Vorhersagemodell nutzen kann.

Als Datenquelle nutze ich derzeit einen CSV-Export aus Grafana. Einen zweiten Weg über die Backend-API habe ich im Skript schon angelegt, er liefert dieselbe Tabellenstruktur.

Gemessen wird an sechs Stellen: drei Beete mit unterschiedlichem Pflanzenkohle-Anteil (Carla 10 %, Berta 5 %, Ilse 0 %), je ein Sensor oben und unten. Jeder Sensor liefert Bodenfeuchte, Bodentemperatur und elektrische Leitfähigkeit.

Am Ende habe ich eine Tabelle mit den Spalten `time`, `bed`, `position`, `moisture`, `temperature`, `ec` und `block`.

## Die vier Verarbeitungsschritte

Mein Skript durchläuft vier Schritte, die nacheinander aufeinander aufbauen.

**1. Einlesen und Umformen** (`lade_daten_csv`): Der Export liegt im Wide-Format vor, also mit einer Spalte je Kombination aus Beet, Position und Messgröße. Ich forme das in ein Long-Format um, bei dem Beet und Position eigene Spalten sind. Dadurch kann ich nach Sensor gruppieren.

**2. Zeitraster vereinheitlichen** (`aufbereiten`): Die Sensoren senden zeitlich versetzt, deshalb waren vor der Aufbereitung rund 36 Prozent der Zellen leer. Mit `resample` lege ich ein Zwei-Stunden-Raster an und mittele die Werte je Zeitfenster. Mit `interpolate` fülle ich anschließend Lücken, aber nur bis zu sechs Stunden. Längere Ausfälle lasse ich bewusst offen, damit keine Werte entstehen, die später nicht mehr von echten Messungen zu unterscheiden wären.

**3. Leeren Anfang abschneiden** (`leeren_anfang_abschneiden`): Der Export (Die letzten 90 Tage) beginnt früher, als Messdaten vorliegen. Alles vor der ersten gültigen Messung entferne ich.

**4. Blöcke bilden** (`bloecke_markieren`): Jeder zusammenhängende Abschnitt ohne Lücke bekommt eine eigene Block-Kennung. Abschnitte unter 48 Stunden werfe ich raus.

## Warum die Blockbildung nötig ist

Für das spätere Training brauche ich vergangene Messwerte als Eingangsgrößen, etwa den Wert von vor 24 Stunden. In pandas bilde ich das über `shift()`, das eine feste Anzahl Zeilen zurückgeht.

Diese Operation zählt Zeilen, nicht Zeit. Über eine Datenlücke hinweg würde sie einen zeitlich weit entfernten Wert liefern, ohne dass es auffällt. Die Block-Kennung verhindert das, weil ich in den folgenden Schritten je Block statt je Sensor rechne.

## Ergebnis

Nach Durchlauf aller Schritte enthält die Tabelle keine fehlenden Werte mehr.

| Kennzahl        | Wert                      |
|-----------------|---------------------------|
| Zeitraum        | 05.08.2026 bis 29.09.2026 |
| Zeitraster      | 2 Stunden                 |
| Sensoren        | 6                         |
| Nutzbare Blöcke | 12 (2 je Sensor)          |
| Fehlende Werte  | 0                         |

Jeder Sensor weist genau zwei Blöcke auf: einen längeren Abschnitt von August bis Anfang September mit 316 bis 462 Messpunkten, und einen kurzen Abschnitt ab dem 22.09. mit je 85 Punkten.

## Auffälligkeit in den Daten

Durch die Blockbildung ist mir ein Datenausfall im September aufgefallen. Die Sensoren fielen zeitlich gestaffelt zwischen dem 01.09. und 13.09. aus und haben die Übertragung nahezu gleichzeitig am 22.09. wieder aufgenommen. Die längste Lücke betrug 514 Stunden.

Ursache war ein Ausfall der Sensoren selbst. Das Problem haben wir gelöst, indem wir die Sensoren besser positioniert haben, sodass sie sich per Solar besser aufladen.

Zusätzlich wollen wir die Senderate auf eine halbe Stunde bis eine Stunde verringern.